In [ ]:
```
Tuple:
It is represnted with open and closed bracket ().
It is immutable (You cant chnage the value ).
It allows duplicate.
It has 2 sub function called as
a. Count b. Index
```

In [1]:
```
t = ()
t
```

Out[1]: ()

In [2]:
```
type (t)
```

Out[2]: tuple

In [3]:
```
t1 = (10,20,30)
t1
```

Out[3]: (10, 20, 30)

In [5]:
```
t2 = (10,2.3,'nit',True,(1+2j))
t2
```

Out[5]: (10, 2.3, 'nit', True, (1+2j))

In [6]:
```
print(t)
print(t1)
print(t2)
```

```
()
(10, 20, 30)
(10, 2.3, 'nit', True, (1+2j))
```

In [7]:
```
t1.count(10)
```

Out[7]: 1

In [8]:
```
t1.index(20)
```

Out[8]: 1

In [9]:
```
t1.append(10)
```

```
---------------------------------------------------------------------
---------
AttributeError                            Traceback (most recent
call last)
Cell In[9], line 1
----> 1 t1.append(10)

AttributeError: 'tuple' object has no attribute 'append'
```

In [10]:
```
t1(10)
```

```
---------------------------------------------------------------------
---------
TypeError                                 Traceback (most recent
call last)
Cell In[10], line 1
----> 1 t1(10)

TypeError: 'tuple' object is not callable
```

In [11]:
```
t1(20) = 200
```

```
  Cell In[11], line 1
    t1(20) = 200
    ^
SyntaxError: cannot assign to function call here. Maybe you meant
'==' instead of '='?
```

In [12]:
```
t1 * 3    # It repeats 3 times(multiplies)
```

Out[12]: (10, 20, 30, 10, 20, 30, 10, 20, 30)

In [ ]:
```
Set():
It is represented with curly brackets {}.
set and dict are assigned with curly brackets {}.
Empty curly brackets {} system automatically understans as dict.
{1,2,3} if values are written in curly brackets system understands as set.
If user assigned any random numbers in set itwill arrange in order. EX s =
{5,8,6,2,4,3,1} its output is {1,2,3,4,5,6,8}
Set takes only one argument.
In set indexing and slicing is not allowed.
set.pop() this will remove random value from the set.
set.pop(2) it doesnt allow indexing.
set is immutable (cant change the value)
Duplicates are not allowed.
```

In [13]:
```
s = {}
s
```

Out[13]: {}

In [14]:
```
type(s)
```

Out[14]: dict

In [16]:
```
s1 = {100,4,10,3,90,20,50} # here it displays in order
s1
```

Out[16]: {3, 4, 10, 20, 50, 90, 100}

In [17]:
```
type(s1)
```

Out[17]: set

In [18]:
```
s2 = {3,1+2j,True,3.4}
s2
```

Out[18]: {(1+2j), 3, 3.4, True}

In [19]:
```
print(s)
print(s1)
print(s2)
```

```
{}
{50, 3, 100, 4, 20, 90, 10}
{3, True, 3.4, (1+2j)}
```

In [21]:
```
s1.add(25) # Here it add in between the value as per sequence of numbers.
s1
```

Out[21]: {3, 4, 10, 20, 25, 50, 90, 100}

In [22]:
```
s3 = s1.copy() # Here it copies the set to s3
s3
```

Out[22]: {3, 4, 10, 20, 25, 50, 90, 100}

In [23]:
```
print(s1)
print(s2)
print(s3)
```

```
{50, 3, 100, 4, 20, 90, 25, 10}
{3, True, 3.4, (1+2j)}
{3, 100, 4, 10, 50, 20, 25, 90}
```

In [24]:
```
s1[0]
```

```
---------------------------------------------------------------------
---------
TypeError                                 Traceback (most recent
call last)
Cell In[24], line 1
----> 1 s1[0]

TypeError: 'set' object is not subscriptable
```

In [25]:
```
s1.pop()
```

Out[25]: 50

In [26]:
```
s1.pop(2) # Here indexing not allowed
```

```
---------------------------------------------------------------------
---------
TypeError                                 Traceback (most recent
call last)
Cell In[26], line 1
----> 1 s1.pop(2) # Here indexing not allowed

TypeError: set.pop() takes no arguments (1 given)
```

In [27]:
```
s1.discard(1000) # it will never give error
                 #  if the value is avaiable in ser it removes or will not
give error.
```

In [28]:
```
s1.remove(1000)
```

```
---------------------------------------------------------------------
---------
KeyError                                  Traceback (most recent
call last)
Cell In[28], line 1
----> 1 s1.remove(1000)

KeyError: 1000
```